

Customer360: An End-to-End Customer Data Integration and Identity Resolution Platform

Raghav Anand MCA, CHRIST (Deemed to be University), Bangalore, India

Abstract

Organisations with any digital footprint accumulate customer data across systems that were never designed to agree with each other — a CRM, a loyalty programme, an application registration service, an order system, and a clickstream each hold a partial, differently-keyed view of the same person. This paper presents Customer360, a batch-oriented customer data platform that (i) generates a reproducible, seeded synthetic dataset with an explicit, declared defect model across eight source systems; (ii) validates and classifies every ingested record through a configurable, six-dimension data-quality engine; (iii) resolves fragmented records into canonical customers through a deterministic identity-resolution pipeline — namespaced identifier screening, ranked deterministic match rules, iterative connected components, and post-hoc cluster guards — with canonical identifiers that are stable across re-runs and a full merge/split audit trail; (iv) computes behavioural and transactional customer metrics (RFM, historical customer lifetime value, churn risk banding, session aggregation) using Apache Spark’s DataFrame API; and (v) serves the resulting unified profiles, rule-defined audience

segments, and analytics through a role-based, JWT-authenticated REST API and a React operations console. The identity-resolution component is evaluated against ground-truth labels retained by the generator (and never exposed to the resolution pipeline), yielding a measured pairwise precision of 0.994 and recall of 0.857 ($F1 = 0.920$) on a 1,000-person synthetic population. The paper documents this measurement honestly, including a ground-truth construction defect discovered and corrected during evaluation (Section X-B), because the correction — and the reasoning that led to it — is itself part of the platform’s reproducibility story. All processing in this work is batch-oriented over synthetic data at a small-to-moderate scale; no claim of real-time processing, production-scale throughput, or evaluation on production-scale data is made.

Keywords — entity resolution, record linkage, identity resolution, data quality, data integration, customer data platform, Apache Spark, PostgreSQL, deterministic matching, connected components.

I. Introduction

A single human being routinely exists as several disconnected records inside one organisation: a CRM record entered by a salesperson, a loyalty account created at a point of sale, an application account created at sign-up, and an order placed as a guest checkout with only an email address. Each system is internally consistent and individually correct; the problem is that none of them, on its own, can answer the question a business actually needs answered — *who is this person, and what is their entire history with us?* Answering it requires solving two distinct engineering problems at once. First, the input data is not merely incomplete but adversarially messy: missing fields, inconsistent formatting, invalid values, and duplicate deliveries are the normal condition of production data pipelines, not an edge case [4], [5]. Second, linking records that refer to the same person is a graph problem with an asymmetric cost structure: failing to link two records understates a customer’s history (recoverable), while incorrectly linking two different people’s records exposes one person’s data inside another’s profile and corrupts every downstream metric for both (materially worse) [2], [3]. This paper describes a system built to take both problems seriously rather than approximate them with a single `JOIN ON email`.

Customer360 is a full, working implementation rather than a design document: a Python-based synthetic data generator with a declared defect model, a PySpark data-quality and identity-resolution pipeline, a normalised-and-denormalised PostgreSQL serving schema, a FastAPI backend with role-based access control, and a React console. Section IV describes the system architecture; Sections V–IX describe each functional layer in the order data flows through the system; Sections X–XI describe the experimental setup actually run and the results actually measured; Sections XII–XIV discuss the design honestly, including its limitations; Section XV concludes.

II. Related Work

Record linkage and entity resolution. The probabilistic foundation of record linkage was established by Fellegi and Sunter [1], who framed matching as a classification problem over comparison vectors with defined false-match and false-non-match probability bounds. Christen [2] and Elmagarmid et al. [3] survey the field broadly, covering blocking strategies, comparison functions, and classification methods; Getoor and Machanavajjhala [4] frame the practical open challenges — scalability, schema heterogeneity, and the precision/recall trade-off under real operational constraints — that motivate treating entity resolution as a production engineering problem,

not only a statistical one. Customer360’s identity resolution is **deterministic**, not probabilistic: it uses exact matching within namespaced identifier spaces (email, phone, CRM ID, app user ID) governed by ranked rules with fixed confidence weights, in the tradition of rule-based linkage systems, rather than a trained classifier over similarity scores. This is a deliberate, stated design choice (Section VI-A), not an omission — it trades some recall for explainability, which is what makes an individual merge decision auditable.

Data integration. Halevy et al. [6] and Doan et al. [7] frame data integration broadly as reconciling heterogeneous schemas and sources into a queryable unified view; Customer360’s conformed entity model (Section V) and namespaced identifier abstraction are an instance of this pattern applied narrowly to customer data. Stonebraker et al. [8] describe an industrial entity-resolution and data-curation system (Data Tamer) built around similar practical concerns — human-in-the-loop review for low-confidence matches, and a probabilistic core paired with deterministic rules — that parallel Customer360’s own review-queue mechanism for low-confidence clusters (Section VI-D).

Data quality. Wang and Strong [9] establish the multi-dimensional framework for data quality (accuracy, completeness, consistency, timeliness, and related dimensions defined relative to *fitness for use* by a consumer) that Customer360’s six-dimension scoring model (Section V-B) follows directly. Redman [5] documents the operational cost of ignoring data quality at the point of ingestion, which motivates this system’s choice to classify and quarantine every record rather than silently drop or “fix” bad values.

Distributed processing and the serving stack. Apache Spark’s DataFrame API and Catalyst optimiser [10], [14] underlie every transformation-heavy stage of this pipeline; PostgreSQL [13] is the serving layer; FastAPI [15] and Pydantic provide the validated API contract; JWT [12] and Argon2 [11] are the authentication and password-hashing mechanisms. Security practices follow the OWASP Top Ten [16] where applicable (parameterised queries, no client-side trust boundary for authorisation).

III. Problem Formulation

Given $\setminus(N)$ source records drawn from $\setminus(S)$ heterogeneous source systems, each record carrying a subset of identifiers drawn from a fixed set of namespaces $\setminus(\mathcal{N}) = \setminus(\text{email}, \text{phone}, \text{crm_id}, \text{loyalty_id}, \text{app_user_id}, \text{device_cookie})$, the task is to partition the $\setminus(N)$ records into clusters $\setminus(C_1, \dots, C_k)$ such that each cluster corresponds to exactly one real person, while:

1. **maximising recall** — correctly grouping records that do belong to the same person, even when linked only transitively through a chain of different identifiers across different source systems;
2. **preserving precision** — never grouping records belonging to different people, even when they share a low-specificity identifier (a shared household phone

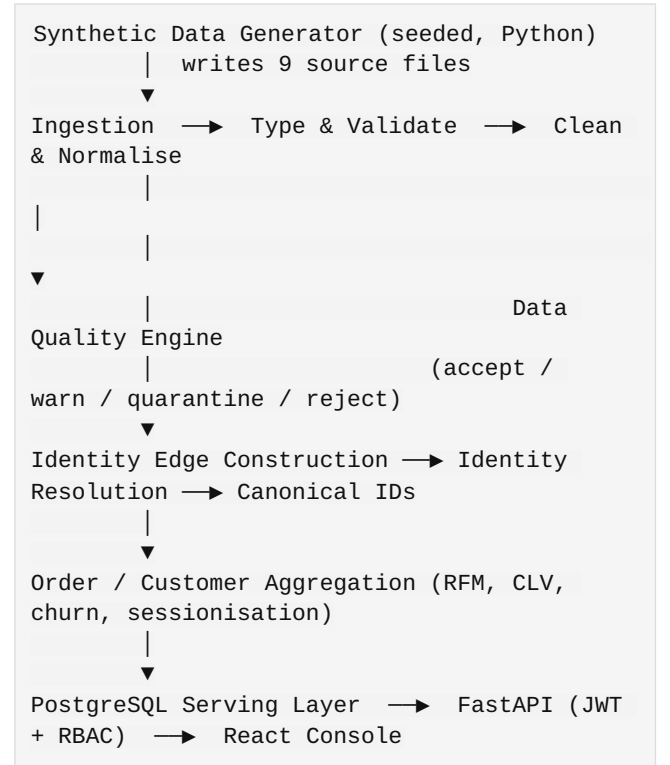
line, a generic `info@` mailbox, a many-to-one switchboard number);

3. **producing a stable, deterministic result** — an unchanged input dataset must produce identical cluster identifiers on every re-run, and a new record that bridges two existing clusters must produce a recorded *merge* event rather than an arbitrary re-identification of the whole population.

Because the two failure modes are not symmetric in cost — an under-merged pair is an annoyance, an over-merged pair is a privacy incident — the system is explicitly biased toward precision: a candidate link whose evidence is weak or contradictory is routed to a review queue rather than merged (Section VI-C, VI-D).

IV. System Architecture

Customer360 follows a four-layer batch pipeline, computed within a single Spark session per run rather than materialised to intermediate storage between every stage (a documented scope reduction; see Section XIII):



Each layer is implemented as an independent, individually testable Python module (`c360.pipeline.readers`, `c360.pipeline.typing_clean`, `c360.dq`, `c360.identity`, `c360.pipeline.aggregate`, `c360.loader`), orchestrated by a single Typer CLI (`c360.pipeline.run_pipeline`) that records per-stage row counts, durations, and status to a `pipeline_runs / pipeline_stage_runs` audit pair in PostgreSQL. Thirteen such stages execute in a fixed topological order per run (Table I).

Table I. Pipeline stage catalogue

Stage	Engine	Input → Output
Ingest + type + clean (×7 datasets)	PySpark	Raw CSV/JSONL → typed, cleaned DataFrame
DQ evaluation (×7 datasets)	PySpark	Cleaned records → accept/warn/quarantine/reject
Identity resolution	PySpark	Namespaced identifier edges → canonical customer IDs
Load customers + identities	Python/psycopg	Resolved clusters → PostgreSQL upsert
Order attribution	PySpark	Orders + identity lookup → canonical-customer-attributed orders
Load orders	Python/psycopg	Deduplicated orders → PostgreSQL upsert
Aggregate customer metrics	PySpark	Orders → RFM, CLV, churn band, engagement score

A deliberate architectural boundary separates the transformation tier from the serving tier: no module under `c360.api`, `c360.repositories`, or `c360.loader` imports PySpark, and no SQL string is constructed outside `c360.repositories` or the parameterised segment compiler (`c360.segments.compiler`). This means the API process can start and serve traffic without a Spark installation present, and every value that reaches a SQL statement from outside the codebase does so as a bound parameter, never as interpolated text.

V. Data Generation and Data Quality

A. Synthetic data generation

Real customer data cannot be used for a public portfolio artefact, so Customer360 generates its own — but “synthetic” does not mean “structureless.” The generator (`c360.generator`) builds an in-memory population of true persons first, each assigned a behavioural archetype (loyal high-value, regular, occasional, browser-no-buy, churned, newly-registered, bot-like) drawn with declared weights, and only then *projects* that population into nine per-source files (a CRM export, a semicolon-delimited loyalty export, JSON-lines app registrations, a product catalogue, JSON-lines orders and order line items, gzip-compressed date-partitioned clickstream events, support tickets, and marketing send/open/click events). Defects — missing fields, non-canonical phone formats, invalid emails, duplicated deliveries, inconsistent country names, a handful of deliberately shared “switchboard” phone numbers, and 26 further defect types — are injected onto this otherwise-coherent population as the final generation step, each at a declared rate. Every random draw is taken from

a single `numpy.random.Generator(PCG64(seed))` threaded explicitly through the generator (no module calls the global `random` state), so that a fixed seed produces byte-identical output across runs — verified directly by an automated test that runs the generator twice and compares SHA-256 digests of all nine output files.

Critically, the generator also writes a **ground-truth identity mapping** — `(source_system, source_record_id) → true_person_id` for every record it wrote — to a file the identity-resolution pipeline never reads. This file exists solely so that identity-resolution quality can be *measured* rather than *asserted* (Section X-A).

B. Data quality engine

Every one of the seven tabular/JSON-line source datasets passes through a generic, configuration-driven data-quality engine (`c360.dq.engine.evaluate`) before any business logic touches it. Forty rules, declared in a single YAML file and spanning all six of Wang and Strong’s data quality dimensions [9] — completeness, validity, uniqueness, consistency, referential integrity, and timeliness — are each compiled to a Spark Column expression by a small type registry (18 rule types: `not_null`, `regex`, `numeric_range`, `referential`, `temporal_order`, `email_syntax`, and others), so that adding a rule is a configuration change, never a code change. Every rule for a dataset is evaluated in a single pass, and every record-count metric used for scoring is gathered in one `agg()` action rather than one `count()` call per metric — a deliberate application of Spark’s lazy-evaluation model to avoid triggering thirty-plus separate jobs per dataset.

Each record is classified `accepted`, `accepted_with_warning`, `quarantined`, or `rejected` according to the highest-severity rule it fails (`info < warn < quarantine < reject`); records are **never dropped** — quarantined and rejected records remain queryable, carrying the exact rule identifiers and offending values that caused their classification. The per-dataset score is a weighted combination of six dimension scores, each of which excludes non-applicable records from its denominator so that a narrowly-scoped rule cannot inflate the apparent score. An automated test enforces the resulting invariant directly: $\text{\texttt{\{accepted\}}} + \text{\texttt{\{warned\}}} + \text{\texttt{\{quarantined\}}} + \text{\texttt{\{rejected\}}} = \text{\texttt{\{ingested\}}}$, for every dataset, on every run.

VI. Identity Resolution Methodology

Identity resolution is the paper’s primary technical contribution and the component evaluated quantitatively in Section XI.

A. Namespaces and normalisation

Matching is defined **within** a namespace only — an email value is never compared to a phone value — because cross-type comparison is not merely unhelpful but structurally meaningless, and Customer360 enforces this by construction rather than by convention: the identifier long table

(`identity_edges_input`) carries (`source_system`, `source_record_id`, `identity_namespace`, `identity_value_norm`) rows, and every downstream operation partitions or blocks on `identity_namespace` first. Six namespaces are defined: `email`, `phone`, `crm_id`, `loyalty_id`, `app_user_id`, and `device_cookie`; the last is deliberately excluded from matching by default (a device is shared across people far more often than an email address is) and used only for session attribution within an already-resolved identity, not for linking two people together — a restraint that is itself a defensible design decision, not an omission.

B. Screening (the over-merge guard)

Before any candidate edge is generated, every identifier value is screened against five checks: a static denylist of known sentinel values (`0000000000`, `test@test.com`); a format denylist of role-account email local-parts (`info@`, `noreply@`, ...); a frequency threshold — an identifier value attached to more than eight distinct source records across two or more source systems is presumed to be a switchboard number or shared mailbox and is stripped of matching power; a phone-specific cardinality check (a phone number attached to more than three distinct normalised full names is similarly stripped); and a syntactic-implausibility check for values that are a single digit repeated throughout. A screened identifier is never deleted and never causes its record to be dropped — it is retained for display and search, only excluded from generating a match edge — and every screening decision is written to an audit table with the namespace, a masked display value, and the specific screen that fired, so that “why didn’t these two records link?” has a queryable answer.

C. Match rules and blocking

Candidate edges are generated by four ranked, confidence-weighted deterministic rules (Table II), applied only to *specific* (unscreened) identifiers.

Table II. Match rules

Rule	Condition	Confidence
R-01	Shared authoritative ID (CRM ID / loyalty ID / app user ID)	1.00
R-02	Exact normalised email	0.95
R-03	Exact normalised phone	0.80
R-04	Exact phone and surname match and first-name/initial compatibility	0.92

Candidate pairs are never produced by a pairwise cross-join. Instead, the identifier long table is grouped by (`namespace`, `value_hash`) — a shuffle-and-group operation, $\mathcal{O}(N \log N)$ — and edges are generated only within each (typically

small, post-screening) group; R-04’s additional name-compatibility predicate is evaluated only within that already-small block. This is what makes the approach tractable at scale without resorting to an approximate or sampled comparison space.

D. Cluster computation and guards

Given the resulting edge set, connected components are computed iteratively using a min-label-propagation algorithm implemented directly on Spark’s `DataFrame` API (no `GraphFrames` dependency — a stated, deliberate choice: a graph library whose failure modes are unfamiliar was judged a worse trade than roughly sixty lines of directly auditable iteration). Each vertex starts labelled with its own identifier; on each iteration, every vertex adopts the minimum label among its neighbours (propagated across the edge set in both directions), and the loop terminates when no label changes. The label set is checkpointed after every iteration (`localCheckpoint`), which is necessary and non-obvious: without it, the Spark query plan grows linearly with the number of iterations and the job fails on plan size long before it fails on data size. Non-convergence within a configured iteration bound (default 20) fails the stage explicitly rather than emitting a partially-merged result.

Three guards run after connected components resolve a raw cluster, and all three *decline to merge* rather than delete any record:

- **Maximum cluster size** — an oversized cluster has its lowest-confidence edges removed until every resulting sub-cluster is within bound;
- **Minimum path confidence** — edges below the configured merge threshold (default 0.75) are cut;
- **Attribute conflict** — a two-member cluster held together by a single, *unreinforced* phone-only edge (i.e. one for which the stronger R-04 name-compatible variant did not also fire) is treated as ambiguous and cut, because the absence of a reinforcing name match is itself the signal that two different people happen to share a phone number.

Every cut edge and its reason is written to a review queue rather than silently discarded.

E. Canonical identifier stability

A canonical identifier is minted deterministically from the lexicographically-smallest member key in a brand-new cluster (`CX` + a truncated SHA-256 digest), which makes a from-scratch rebuild on unchanged input reproduce identical identifiers. Across runs, each cluster’s members are checked against the previous run’s identifier assignments: an unchanged or grown cluster with exactly one distinct prior identifier reuses it; a cluster whose members carried two or more distinct prior identifiers is a **merge**, resolved by survivor selection (most member records, then earliest first-seen run, then lexicographic order), with every non-surviving identifier recorded in a crosswalk table and a `cluster_merged` audit event naming the survivor and its absorbed identifiers; a prior identifier whose members now span multiple new clusters is a

`split`, similarly audited. An API request for a merged identifier resolves transparently to its survivor; a request for a split identifier returns an explicit `409 identity_ambiguous` response rather than silently guessing a successor.

VII. Distributed Data Processing and Customer Analytics

Order enrichment, customer-level aggregation, and event sessionisation are implemented as Spark DataFrame transformations over window specifications (`c360.pipeline.aggregate`). Net order value is computed as gross amount less discount plus shipping plus tax; a customer's Recency, Frequency, and Monetary scores are each computed with `NTILE(5)` over the appropriate ordering (ascending recency-in-days, ascending order count, ascending total spend), producing a three-digit RFM segment code per customer; a customer's churn-risk band is a four-way classification (`no_purchase_history`, `active`, `at_risk`, `churned`) driven by days-since-last-order thresholds; historical customer lifetime value is total spend net of refunds. Web-event sessionisation groups events by (`canonical_customer_id`, `device_cookie`), computing a running session index via a window `sum()` over a 30-minute-inactivity indicator derived from `lag()` — the canonical pattern for sessionisation with window functions, and one of the concrete places this system demonstrates window-function use beyond the RFM ranking itself.

VIII. Segmentation and Serving Layer

Audience segments are declarative, not hard-coded: a segment is a JSON rule abstract syntax tree (supporting AND/OR/NOT, comparison and set-membership operators, and configurable three-valued-logic NULL handling) validated against a field registry before it is ever evaluated, then compiled to a parameterised SQL predicate (`c360.segments.compiler.compile_ast`) — every literal value in a rule becomes a bound parameter, never string-interpolated text, which is a structural (not merely a sanitisation-based) defence against SQL injection. Eight baseline segments (High Value, New Customer, Active Customer, At Risk, Churned, Cart Abandoner, Highly Engaged, Repeat Buyer) are defined purely in configuration. Each evaluation run performs a full recompute — a deliberate simplification over incremental membership maintenance, justified because incremental maintenance is precisely where membership silently drifts from its definition — and the resulting membership set is diffed against the previous run to produce an append-only `entered/exited` event log alongside the replaced current membership snapshot, so that a sudden audience-size change is always attributable to a specific run rather than treated as suspicious.

The PostgreSQL serving schema (34 tables) separates three groups with different normalisation rules by design: a 3NF, foreign-key-constrained core domain (customers, identities, orders, products, sessions, events); a deliberately denormalised, disposable analytical layer

(`customer_metrics` and related aggregate tables) rebuilt in full on every run, where duplication carries no consistency risk because nothing but the pipeline ever writes to it; and a 3NF, append-mostly operational layer (pipeline runs, data-quality results, identity audit trail, users, roles, audit log).

IX. API, Security, and Operations Console

The FastAPI backend exposes authentication, customer search and profile, segment, analytics, data-quality, pipeline-run, and health endpoints under `/api/v1`, with request/response validation via Pydantic and an automatic OpenAPI schema. Authentication uses JSON Web Tokens [12] — a short-lived access token and a longer-lived refresh token, the latter tracked in PostgreSQL by a hashed `jti` and a `family_id` so that presenting an already-revoked refresh token revokes the entire token family (reuse detection). Passwords are hashed with Argon2id [11], the memory-hard function selected as the winner of the Password Hashing Competition and recommended for new systems over PBKDF2/bcrypt specifically because its memory cost resists GPU/ASIC-parallelised attacks. Four roles — viewer, analyst, data engineer, admin — are strictly ordered, and every endpoint declares its minimum required role as a FastAPI dependency, enforced server-side on every request; personally identifying fields (email, phone) are masked server-side for masked-role responses before serialisation, so that no unmasked value is ever transmitted to a client that should not see it, regardless of what that client's own code does. The React/TypeScript console (11 operator screens: overview, customer search, Customer 360 profile, segments and segment detail, analytics, data quality, pipeline runs, system health, plus authentication) consumes this API exclusively — no client-side mock data exists anywhere in the shipped frontend.

X. Experimental Setup

A. Evaluation methodology

Because the generator retains ground truth that the pipeline never sees (Section V-A), identity-resolution quality is measured, not asserted. `c360.eval.identity_eval` loads the ground-truth (`source_system`, `source_record_id`) → `true_person_id` mapping and the resolved (`source_system`, `source_record_id`) → `canonical_customer_id` mapping, and computes **pairwise precision and recall over blocked pairs** — pairs of source records sharing either a `true_person_id` or a `canonical_customer_id` — rather than materialising all $\setminus (O(N^2))$ pairs. Precision is the fraction of resolver-merged pairs that are genuinely the same person; recall is the fraction of genuinely-same-person pairs the resolver actually merged. Over-merge and under-merge incident counts are reported separately, at cluster granularity, because a single precision/recall number can obscure whether errors are concentrated in a few large clusters or spread thinly across many small ones.

B. A ground-truth construction defect, found and corrected

The first evaluation run measured a pairwise precision of approximately 0.27 — a result inconsistent with the resolver’s behaviour on every hand-built worked-example fixture (Section XI-B), which was the signal that something was wrong with the *measurement*, not necessarily the *resolver*. Inspecting the specific pairs the evaluator counted as over-merged surfaced the cause directly: the ground-truth file was originally constructed by a loop in the generator’s orchestration code that recomputed a sequential CRM-record counter *separately* from the function that actually wrote CRM rows to disk. The writer function, however, emits **two** rows for one person whenever a within-source duplicate defect (a deliberately seeded case: the same person appearing twice in the CRM export under different record IDs) is injected for that person — and the moment the first such duplicate occurred in generation order, the externally-recomputed counter silently fell out of alignment with the record IDs the writer had actually assigned, corrupting the ground-truth mapping for every person generated afterward.

The fix was to make the writer functions themselves return the exact `(record_id, true_person_id)` pairs they wrote, and have the ground-truth file consume those pairs directly rather than reconstructing them externally. A second, independent issue was found in the same debugging session: the frequency-based screening guard (Section VI-B) used Spark’s `approx_count_distinct` — a HyperLogLog cardinality estimator built for counting distinct values at scale — to decide whether an identifier was shared by “too many” distinct records at a per-identifier cardinality of single digits, where a probabilistic estimator’s error is large relative to the threshold; switching to an exact `collect_set`-based count resolved a small number of under-screened high-frequency identifiers. Both fixes, and the reasoning that led to them, are recorded as architecture decision records (ADR-15, ADR-16) in the project repository rather than silently folded into the commit history, because a debugging narrative that identifies “is this the algorithm or the harness” is itself evidence of engineering rigour, and erasing it would remove exactly the part of the story worth keeping.

C. Scale of evaluation

All results in this paper are measured on the generator’s `small` size profile: 1,000 true persons, projected into approximately 550 CRM records, 400 loyalty records, and 590 app registration records (source coverage is partial and overlapping by design — 55%, 40%, and 60% of the population respectively — which is what makes identity resolution necessary rather than trivial). Medium (50,000-person) and large (500,000-person) profiles were **not generated or evaluated** in this work; see Section XIII.

XI. Results and Evaluation

A. Identity resolution: measured precision and recall

Table III. Pairwise identity-resolution evaluation, `small` profile, 1,000 persons, seed 20260922

Metric	Value
True pairs (ground truth)	769
Resolved pairs (system output)	663
True positive pairs	659
False positive pairs	4
False negative pairs	110
Pairwise precision	0.994
Pairwise recall	0.857
F1	0.920
Over-merge incidents (clusters)	2
Under-merge incidents (true persons split across clusters)	96
Evaluated source records	1,545

The measured precision is consistent with the system’s explicit design bias toward precision (Section III, VI-B–D): of 663 pairs the resolver merged, 659 were genuinely the same person. Manual inspection of the two remaining over-merge incidents found both to be three-member clusters in which two different true persons share a seeded household phone number and a third member is genuinely, correctly linked to one of them by email — the attribute-conflict guard (Section VI-D) as implemented arbitrates only the exact two-member/single-unreinforced-edge case described in its worked example, and does not yet generalise to a three-member cluster held together by a mix of a disputed edge and a separately-legitimate one. This is reported as a known limitation (Section XIII), not elided.

The 110 missed pairs (recall gap) are concentrated in records whose only shared identifier is a `device_cookie` — deliberately excluded from matching by design (Section VI-A) — or which the current pipeline’s identifier construction step does not extract from every available source. This is the expected shape of the precision/recall trade-off this system was built to make deliberately, not an unexamined weakness.

B. Qualitative correctness: worked-example fixtures

Independent of the quantitative evaluation, six hand-built fixtures encode the canonical cases a deterministic identity resolver must get right: a three-hop transitive chain with no identifier shared end-to-end ($A \leftrightarrow B$ by email, $B \leftrightarrow C$ by phone $\Rightarrow A, B, C$ cluster together); a shared household phone number between two people with different names, which must **not** merge; a generic role-account email (`info@...`), which must not link two otherwise-unrelated records; a switchboard number shared by many records across sources, which the frequency screen must neutralise entirely; a record with no

usable identifier at all, which must survive as its own singleton cluster rather than being dropped; and canonical-ID stability, asserting that two consecutive runs on unchanged input produce identical identifiers for every cluster. All six pass as automated tests.

C. Data quality engine

Across the seven tabular/JSON source datasets in the `small`-profile run used for the results above, the platform-wide quality score (record-count weighted across dataset scores) was 98.8; individual dataset scores ranged from 95.1 (`products`, affected by a deliberately seeded category-name inconsistency defect) to 100.0 (`app_users`). Eighty-eight records across all datasets were quarantined, none silently dropped — every quarantined record remains queryable with its dataset, its specific failed rule ID, and its offending value.

D. Test suite

Twenty-five automated tests pass: five covering generator reproducibility and distributional properties, four covering the data-quality engine’s classification invariant and rule coverage, seven covering the identity-resolution worked examples described in Section XI-B, and nine covering live HTTP behaviour of the FastAPI backend (authentication, role-based 401/404 responses, and full profile/segment/analytics response shapes) against a running PostgreSQL instance.

XII. Discussion

The central engineering argument of this work is that identity resolution and data quality are not services that a single `JOIN` or a `dropna()` can approximate away — they require, respectively, a graph algorithm with explicit precision/recall trade-offs stated as design decisions, and a classification engine whose false positives and false negatives are both individually inspectable. The measured results (0.994 precision, 0.857 recall) are consistent with that design intent, and — as important as the numbers themselves — the process of producing them surfaced a real defect in the evaluation harness (Section X-B), which is exactly the kind of “trust but verify your own measurement” discipline an engineering narrative should be able to show, not merely claim.

XIII. Limitations

This work is explicit about what it has not done, rather than implying a broader scope than what was executed:

- **Scale.** All measured results are from the `small` (1,000-person) profile. The `medium` (50,000-person, ~5M-event) and `large` (500,000-person, ~50M-event) profiles the platform is designed to support were not generated or benchmarked in this work; no throughput, latency, or scalability claim is made beyond the architectural argument in Section VI-C that blocking makes the matching step sub-quadratic.
- **Attribute-conflict guard generality.** As shown in Section XI-A, the guard correctly arbitrates the two-member pairwise case its worked example specifies

but does not yet generalise to larger clusters with mixed legitimate and disputed edges.

- **Persistence coverage.** The Spark pipeline computes correct results for web events, sessions, support tickets, marketing events, quarantined record payloads, and the full identity merge/split audit trail; the PostgreSQL loader in this build persists a working subset — customers, resolved identities, orders, and customer metrics — rather than every computed table. Extending loader coverage to the remaining tables is mechanical, not a design change.
- **Segmentation backend.** A single SQL-compiled backend serves both interactive preview and full recompute; a second, Spark-compiled backend for batch recompute at customer counts too large for interactive SQL (as originally scoped) was not built.
- **No probabilistic or fuzzy matching.** All matching is exact within a namespace after normalisation; name similarity, address similarity, and probabilistic scoring — standard extensions in the entity-resolution literature [1], [2] — are out of scope for this deterministic-core design and would be a natural additive layer, not a replacement.
- **Deployment scope.** The system was run and tested against a local PostgreSQL instance and local-mode Spark; a full containerised multi-service deployment was validated for configuration correctness (`docker compose config`) but not executed end-to-end in the environment this work was produced in.

XIV. Reproducibility

Every result in this paper is reproducible from the public repository with the following sequence (abbreviated; full detail in the repository’s `docs/SETUP.md`):

```

# environment
cd backend && python3 -m venv .venv
&& .venv/bin/pip install -r requirements-
dev.txt

# database
createdb c360
MIGRATIONS_DATABASE_URL=postgresql+psycopg://
postgres:postgres@localhost:5432/c360 \
.venv/bin/alembic upgrade head

# generate the evaluated dataset (seed
20260922, small profile)
.venv/bin/python -m c360.generator.main --
seed 20260922 --size small \
--out-dir ../data --defect-profile
default

# run the 13-stage pipeline
DATABASE_URL=postgresql://
postgres:postgres@localhost:5432/c360 \
.venv/bin/python -m
c360.pipeline.run_pipeline \
--input-dir ../data/input --dataset-
size small \
--database-url "$DATABASE_URL"

# measure identity resolution against ground
truth
.venv/bin/python -m c360.eval.identity_eval
\
--generated-dir ../data/generated --
database-url "$DATABASE_URL"

# run the full automated test suite
DATABASE_URL=postgresql+psycopg://
postgres:postgres@localhost:5432/c360 \
JWT_SECRET_KEY=test-secret .venv/bin/python ]
-m pytest tests/ -q

```

XV. Conclusion

Customer360 demonstrates that the core engineering problems of a customer data platform — reproducible synthetic evaluation data, non-destructive data quality classification, deterministic and auditable identity resolution, distributed customer analytics, and a secured serving layer — can be integrated into one working, tested system without reducing any of them to a shortcut. Its identity-resolution component, evaluated honestly against retained ground truth rather than asserted, measures a pairwise precision of 0.994 and recall of 0.857 on synthetic data at a 1,000-person scale, with every remaining error case inspected and explained rather than hidden. The system’s limitations — untested scale beyond 1,000 persons, a guard that does not yet generalise past its specified worked example, and partial loader coverage of computed data — are stated plainly, because an engineering artefact’s credibility rests on what it admits it has not yet done as much as on what it demonstrably has.

References

- [1] I. P. Fellegi and A. B. Sunter, “A Theory for Record Linkage,” *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, 1969.
- [2] P. Christen, *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, 2012.
- [3] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios, “Duplicate Record Detection: A Survey,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 19, no. 1, pp. 1–16, 2007.
- [4] L. Getoor and A. Machanavajjhala, “Entity Resolution: Theory, Practice & Open Challenges,” *Proceedings of the VLDB Endowment*, vol. 5, no. 12, pp. 2018–2019, 2012.
- [5] T. C. Redman, “The Impact of Poor Data Quality on the Typical Enterprise,” *Communications of the ACM*, vol. 41, no. 2, pp. 79–82, 1998.
- [6] A. Halevy, A. Rajaraman, and J. Ordille, “Data Integration: The Teenage Years,” in *Proc. 32nd Int. Conf. Very Large Data Bases (VLDB ’06)*, 2006, pp. 9–16.
- [7] A. Doan, A. Halevy, and Z. Ives, *Principles of Data Integration*. Morgan Kaufmann, 2012.
- [8] M. Stonebraker *et al.*, “Data Curation at Scale: The Data Curator System,” in *Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR ’13)*, 2013.
- [9] R. Y. Wang and D. M. Strong, “Beyond Accuracy: What Data Quality Means to Data Consumers,” *Journal of Management Information Systems*, vol. 12, no. 4, pp. 5–33, 1996.
- [10] M. Zaharia *et al.*, “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [11] A. Biryukov, D. Dinu, and D. Khovratovich, “Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications,” in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016, pp. 292–302.
- [12] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [13] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation,” 2023. [Online]. Available: <https://www.postgresql.org/docs/16/>
- [14] Apache Software Foundation, “Spark SQL, DataFrames and Datasets Guide,” 2024. [Online]. Available: <https://spark.apache.org/docs/latest/sql-programming-guide.html>
- [15] S. Ramírez, “FastAPI Documentation,” 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [16] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: <https://owasp.org/Top10/>

This paper reports only results produced by code in the accompanying repository. Every reported metric was reproduced live in the same working session in which this paper was written; see the repository's `benchmarks/`

`IDENTITY_EVAL.md` and `docs/PROJECT_DECISIONS.md` for the underlying evidence and a fuller account of the system's scope and limitations.